Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería
Sistemas Operativos 2



MANUAL TÉCNICO- PROYECTO FINAL REDXela

Registro académico: 201831260
Nombre: Michael Kristopher Marín Reyes

Quetzaltenango, Octubre de 2026.

Índice

Arquitectura **4**
Stack Tecnológico **4**
Requisitos del Sistema **5**
Diagrama de clases **5**
Dependencias **5**
Estructura del Backend **6**
Módulo: catalog 6
Clases 6
Módulo: customers 6
Clases 7
Módulo: orders 7
Clases 7
Módulo: resources 8
Clases 8
Módulo: warehouse 8
Clases 9
Módulo: inventory 9
Clases 9
Módulo: receipts 10
Clases 10
Módulo: simulation 11
Clases 11
Módulo: status 11
Clases 11
Módulo: metrics 12
Clases 12
Estructura del Frontend **12**
Módulo App.vue 12
App 12
Módulo main.ts 13
Módulo modules 13
ProductCatalog 13
store 13
types 13
api 13
CustomerForm 13
store 14
types 14
InventoryTable 14
WarehouseMap 14

store..... 14
types..... 14
api..... 14
OrderForm..... 14
OrderList..... 15
OrderProgress..... 15
store..... 15
types..... 15
api..... 15
ReceiptPanel..... 15
types..... 15
SimulationControls..... 16
StatusDashboard..... 16
store..... 16
types..... 16
Módulo shared..... 16
http..... 16
NavBar..... 16
serviceLevels..... 17
Acceso al repositorio con el código..... 17

Manual Técnico

El presente manual técnico tiene como finalidad documentar la arquitectura, el stack tecnológico, las dependencias, la configuración, la instalación y la forma de ejecutar en ambiente local el sistema simulador REDXela.

Cubre la configuración del entorno de desarrollo, estructura del proyecto, módulos backend y frontend, mecanismos de concurrencia, comunicación entre componentes, pruebas unitarias e integración y ejecución local del sistema completo.

Arquitectura

El sistema está compuesto por dos aplicaciones independientes que se comunican mediante HTTP REST.

- Backend: Aplicación Spring Boot.
- Frontend: Aplicación Vue 3 con Vite.
- Comunicación: REST para operaciones CRUD para actualizaciones en tiempo real del estado de la simulación.

Stack Tecnológico

Para backend:

- Lenguaje: Java 17.
- Framework: Spring Boot 4.1.1
- Build Tool: Maven.
- Validación: Jakarta Validation
- Concurrencia: java.util.concurrent para ExecutorService, BlockingQueue, Semaphore, rentrantLock.

Para frontend:

- Framework: Vue 3.
- Build Tool: Vite 5.
- Lenguaje: TypeScript.
- Estado: Pïnia.
- HTTP: Fetch API.

Requisitos del Sistema

- Java Development Kit (JDK) 22.
- node.js v20.x.0 o superior
- npm v10.x.o superior

Verificación en el sistema:
java -version
node -v
npm -v

Diagrama de clases

El diagrama se muestra a continuación, sin embargo, adjunto a este documento se encuentra en formato .svg para tener una visualización más detallada.

Dependencias

Para este proyecto se utilizó las siguientes dependencias:
Dependencias para backend:

- spring-boot-starter-web
- spring-boot-starter-validation
- spring-boot-starter-test
- spring-boot-configuration-processor

Dependencias para frontend:

- vue ^3.5.x
- pinia ^2.2.x
- vite ^5.4.x
- typescript ^5.6.x
- vue-tsc ^2.1.x
- @vitejs/plugin-vue

Estructura del Backend

La estructura del backend se define a continuación, se describen clases, propósitos, dónde se usan y métodos principales.

Módulo: catalog

Este módulo se encarga de la gestión del catálogo de productos del centro de distribución. Se utiliza en las clases: OrderService, InventoryService, ReceiptService, OrdersStore/CatalogStore (frontend).

Clases

Product
Entidad que representa un producto. Campos: id, name, merchandiseType, unitVolume. Se utiliza en las clases: CatalogService, Order, Inventory.
Métodos: Getters/Setters, constructores.

MerchandiseType (enum)

Se encarga de la clasificación de mercancía (GENERAL, FRAGILE, HEAVY, BULKY, ELECTRONICS). Se utiliza en las clases Product, filtros/lógica de almacén.

CatalogService
Servicio que mantiene y provee el catálogo. Se usa en las clases: OrderService, InventoryService, ReceiptService, CatalogController. Métodos: register(Product), findById(String), all() - Colección de productos.

CatalogController
Para qué: Expone REST /api/products. Se utiliza en el frontend CatalogStore. Métodos: list() GET /api/products.

Módulo: customers

Se encarga de la gestión de clientes y su nivel de servicio. Se utiliza en OrderService, Orders (frontend).

Clases

Customer
Es la entidad cliente. Campos: id, name, serviceLevel. Se usa: CustomerService, Order.

CustomerService

Sirve para registro y consulta de clientes. Se usa: OrderService, CustomerController. Métodos register(Customer), all(), find(String id).

CustomerController
Es un REST /api/customers. Se usa en el frontend CustomersStore. Métodos: create(request), list().

Módulo: orders

Se encarga de la creación, seguimiento y ciclo de vida de pedidos. Se usa en SimulationEngine, StatusService, OrdersStore (frontend).

Clases

Order
Representa un pedido. Campos: id, customerId, productId/lines, quantity, priority, status, stage, assignedWorkerId, reservationId, timestamps. Se usa: OrderRepository, SimulationEngine, StatusResponse, MetricsService. Métodos: getPriority(), setPriority(int), getters/setters.

OrderStatus (enum)
Contiene estados: CREATED, WAITING_STOCK, QUEUED, WAITING_RESOURCES, PROCESSING, DEADLOCKED, COMPLETED, ERROR.

OrderStage (enum)
Contiene las etapas operativas: NOT_STARTED, PREPARING, SCANNING, LOADING, COMPLETED.

OrderRepository (concurrente)
Se encarga del almacenamiento concurrente de pedidos con ReentrantLock. Se usa: OrderService,

SimulationEngine, StatusService, AdminController. Métodos: all(), find(id), enqueue(Order), awaitNextEligible(snapshot), nextForProcessing(), waitingOrders(), revision().

OrderService
Se encarga de la lógica de creación y consulta de pedidos. Se usa en: SimulationEngine, StatusService, OrdersController. Métodos: create(customerId, productId, quantity), list(page,size), summarize().

OrdersController

Es un REST /api/orders. Se usa en: Frontend OrdersStore. Métodos: create(request), list(page,size), summary().

Módulo: resources

Se encarga del control de recursos compartidos con sincronización. Se usa en: SimulationEngine, ReceiptService, StatusService.

Clases

ResourceType (enum)
Contiene tipos: WORKER, FORKLIFT, PACKING, QUALITY, SCANNER, LOADING.

ResourceInstance
Instancia asignada (id, ownerId, acquiredAt).Se usa: ResourceManager.

ResourceManager

Asigna y libera recursos con Semaphore (justo), ReentrantLock y Condition. Se usa: SimulationEngine, ReceiptService, StatusService. Métodos: acquire(operationId, required) throws InterruptedException, release(operationId), snapshot(), pendingRequests().

ResourceSummary / ResourceRequest
DTOs para snapshot de recursos.

Módulo: warehouse

Se encarga del almacenamiento no contiguo con ubicaciones y lotes. Se usa en: InventoryService.

Clases

Location
Ubicación física (id, capacity, used).

Lot
Lote de mercancía (id, productId, units, allocations, receiptId).

LotAllocation
Asignación lote-ubicación (locationId, units).

Warehouse

Gestión de colocación/retirada de lotes en ubicaciones. Dónde se usa: InventoryService. Métodos: place(productId,units,receiptId), withdraw(lines), tryPlacePending(), locations(), lots().

Módulo: inventory

Inventario con escritor único y reservas atómicas. Se usa en: SimulationEngine, OrderService, ReceiptService, StatusService, AdminController.

Clases

InventoryItem
Estado por producto (productId, stock, reserved, pendingPlacement).

StockReceipt / ReceiptPlacement
Recepciones y su colocación pendiente.

StockReservation
Reserva de stock para un pedido (id, orderId, productId, units, locationLines, lots).

StockIssue
Resultado de consumo de reserva con lotes retirados.

InventorySnapshot

Instantánea inmutable para lecturas consistentes. Métodos: canSatisfy(productId,units), blockingProduct(productId).

InventoryService
Única vía para modificar inventario (ReentrantLock). Se usa: SimulationEngine, ReceiptService, StatusService, AdminController. Métodos: receive(productId,units), applyReceipt(message), planWithdrawal(productId,units), consume(reservationId), recover(reservationId), snapshot(), reset().

InventoryController
REST /api/inventory, /api/warehouse. Se usa: Frontend InventoryStore.

Módulo: receipts

Recepción de mercancía con patrón productor-consumidor. Se usa en: SimulationCoordinator, StatusService.

Clases

ReceiptStatus (enum)
Maneja los estados: PENDING, WAITING_SCANNER, SCANNING, WAITING_INVENTORY, WAITING_SPACE, COMPLETED, ERROR.

CreateReceiptRequest
DTO de entrada (productId, quantity).

ReceiptView
Vista para frontend con estado, placedUnits, pendingUnits.

ReceiptSettings
Parámetros: queueCapacity, maxOutstanding, offerTimeoutMilliseconds, scanSecondsPerUnit.

ReceiptReport
Reporte con colas, totales y lista de ReceiptView.

ReceiptService
Gestiona colas BlockingQueue, workers, coordinación con recursos/inventario.Se usa para: ReceiptController, SimulationCoordinator, StatusService, AdminController. Métodos: submit(key,productId,quantity), start(), stop(), find(id), report().

ReceiptController
REST /api/receipts. Se usa en: Frontend ReceiptPanel. Métodos: create(request,key), list(), find(id).

Módulo: simulation

Motor de simulación y coordinación. Se usa en: StatusService, HealthController, AdminController.

Clases

SimulationSettings
Parámetros de simulación (simulatedSecondsPerRealSecond, workerCount).

SimulationClock
Control de tiempo simulado/real. Métodos: start(), stop(), elapsedMinutes(), realNanos(simulatedSeconds).
SimulationState
Estado compartido con sincronización.

SimulationEngine
Pool de hilos, ciclo de procesamiento de pedidos, coordinación con recursos/inventario/métricas. Se usa en: SimulationCoordinator, StatusService, HealthController, AdminController. Métodos: start(), stop(), status().

SimulationCoordinator
Coordina motor y recepciones. Métodos: start(), stop().

SimulationController
REST /api/simulation/start, /api/simulation/stop.

Módulo: status

Propósito: Agregación de estado y respuestas para dashboard.

Clases

StatusResponse
Respuesta completa (contractVersion, runId, sequence, simulationStatus, resources, warehouse, inventory, orders, activeOrders, waitingOrders, receipts, metrics).

StatusService
Consolida estado de todos los módulos. Se usa en: StatusController, StatusEventService. Métodos: getStatus().

StatusController
REST /api/status.

Módulo: metrics

Métricas de desempeño con aging.

Clases

MetricsService
Acumula contadores con AtomicLong, aplica aging y cálculos. Se usa en: SimulationEngine,

StatusService.
Métodos: recordCompleted(order), recordFailed(order), applyAging(waitingOrders), getSnapshot().

MetricsSnapshot
Métricas calculadas (completed, failed, avgWaitSeconds, avgProcessSeconds, maxWaitSeconds, maxProcessSeconds, throughputPercent).

MetricsController
REST /api/metrics. Se usa en: Frontend MetricsPanel.

Módulos: health, admin, events
HealthController (/api/health): Verifica estado de componentes. Usa SimulationEngine, ReceiptService, InventoryService.

AdminController (/api/admin/snapshot): Genera snapshot completo. Usa InventoryService, OrderRepository, OrderService, ReceiptService, SimulationEngine.

StatusEventService: Publica SSE en /api/events usando ScheduledExecutorService y ThreadPoolExecutor para envío a clientes.

Estructura del Frontend

Módulo App.vue

El módulo App.vue contiene componentes Vue y stores de Pinia que gestionan la interfaz de usuario y la comunicación con el backend.

App

El archivo App.vue forma parte del módulo App.vue. Define comportamiento de UI o estado de aplicación.
Archivo sin exportaciones detectadas por el análisis estático simple.

Módulo main.ts

El módulo main.ts contiene componentes Vue y stores de Pinia que gestionan la interfaz de usuario y la comunicación con el backend.
main
El archivo main.ts forma parte del módulo main.ts. Define comportamiento de UI o estado de aplicación. Archivo sin exportaciones detectadas por el análisis estático simple.

Módulo modules

El módulo modules contiene componentes Vue y stores de Pinia que gestionan la interfaz de usuario y la comunicación con el backend.

ProductCatalog

El archivo ProductCatalog.vue forma parte del módulo modules. Define comportamiento de UI o estado de aplicación. Archivo sin exportaciones detectadas por el análisis estático simple.

store

El archivo store.ts forma parte del módulo modules. Define comportamiento de UI o estado de aplicación.
Exportaciones / funciones detectadas: useCatalogStore

types

El archivo types.ts forma parte del módulo modules. Define comportamiento de UI o estado de aplicación.
Exportaciones / funciones detectadas: merchandiseLabels

api

El archivo api.ts forma parte del módulo modules. Define comportamiento de UI o estado de aplicación.
Exportaciones / funciones detectadas:

- fetchCustomers
- createCustomer

CustomerForm

El archivo CustomerForm.vue forma parte del módulo modules. Define comportamiento de UI o estado de aplicación. Archivo sin exportaciones detectadas por el análisis estático simple.

store

El archivo store.ts forma parte del módulo modules. Define comportamiento de UI o estado de aplicación.
Exportaciones / funciones detectadas: useCustomersStore

types

El archivo types.ts forma parte del módulo modules. Define comportamiento de UI o estado de aplicación.
Archivo sin exportaciones por el análisis estático simple.

InventoryTable

El archivo InventoryTable.vue forma parte del módulo modules. Define comportamiento de UI o estado de aplicación. Archivo sin exportaciones por el análisis estático simple.

WarehouseMap

El archivo WarehouseMap.vue forma parte del módulo modules. Define comportamiento de UI o estado de aplicación. Archivo sin exportaciones por el análisis estático simple.

store

El archivo store.ts forma parte del módulo modules. Define comportamiento de UI o estado de aplicación.
Exportaciones / funciones: useInventoryStore

types

El archivo types.ts forma parte del módulo modules. Define comportamiento de UI o estado de aplicación. Exportaciones / funciones: merchandiseTypeLabels

api

El archivo api.ts forma parte del módulo modules. Define comportamiento de UI o estado de aplicación.
Exportaciones / funciones:

- fetchOrders
- createOrder

OrderForm

El archivo OrderForm.vue forma parte del módulo modules. Define comportamiento de UI o estado de aplicación. Archivo sin exportaciones por el análisis estático simple.

OrderList

El archivo OrderList.vue forma parte del módulo modules. Define comportamiento de UI o estado de aplicación. Archivo sin exportaciones por el análisis estático simple.

OrderProgress

El archivo OrderProgress.vue forma parte del módulo modules. Define comportamiento de UI o estado de aplicación. Archivo sin exportaciones por el análisis estático simple.

store

El archivo store.ts forma parte del módulo modules. Define comportamiento de UI o estado de aplicación. Exportaciones / funciones: useOrdersStore

types

El archivo types.ts forma parte del módulo modules. Define comportamiento de UI o estado de aplicación. Exportaciones / funciones:

- orderStatusLabels
- orderStageLabels

api

El archivo api.ts forma parte del módulo modules. Define comportamiento de UI o estado de aplicación. Exportaciones / funciones:

- createReceipt

ReceiptPanel

El archivo ReceiptPanel.vue forma parte del módulo modules. Define comportamiento de UI o estado de aplicación. Archivo sin exportaciones por el análisis estático simple.

types

El archivo types.ts forma parte del módulo modules. Define comportamiento de UI o estado de aplicación. Exportaciones / funciones:

- receiptStatusLabels

SimulationControls

El archivo SimulationControls.vue forma parte del módulo modules. Define comportamiento de UI o estado de aplicación. Archivo sin exportaciones por el análisis estático simple.

StatusDashboard

El archivo StatusDashboard.vue forma parte del módulo modules. Define comportamiento de UI o estado de aplicación. Archivo sin exportaciones por el análisis estático simple.

store

El archivo store.ts forma parte del módulo modules. Define comportamiento de UI o estado de aplicación.
Exportaciones / funciones:

- useStatusStore

types

El archivo types.ts forma parte del módulo modules. Define comportamiento de UI o estado de aplicación.
Exportaciones / funciones:

- simulationStatusLabels
- resourceLabels

Módulo shared

El módulo shared contiene componentes Vue y stores de Pinia que gestionan la interfaz de usuario y la comunicación con el backend.

http

El archivo http.ts forma parte del módulo shared. Define comportamiento de UI o estado de aplicación.
Exportaciones / funciones:

- apiBaseUrl
- errorMessage

NavBar

El archivo NavBar.vue forma parte del módulo shared. Define comportamiento de UI o estado de aplicación. Archivo sin exportaciones por el análisis estático simple.

serviceLevels

El archivo serviceLevels.ts forma parte del módulo shared. Define comportamiento de UI o estado de aplicación. Exportaciones / funciones:

- serviceLevels
- serviceLevelLabel

Acceso al repositorio con el código

Link del repositorio